2026 春季人工智能导论

项目中期报告

基于 ViT 与 CNN 在 CIFAR-10 上的图像分类性能对比研究

团队成员与任务分工

姓名	主要负责模块与分工内容
黄泽皓	主导数据与训练 Pipeline。负责数据加载、预处理与增强，编写模型训练循环及指标评测逻辑。
张一	主导模型核心实现。负责基于 PyTorch 从零搭建 Vision Transformer 的核心组件，构建对比基线 CNN 模型。
刘梵佑	主导实验调度、可视化与报告撰写。负责编写自动化实验脚本，实现 Attention Map 提取及图表生成。

1 任务与数据说明

1.1 问题定义

本项目旨在完成 CIFAR-10 数据集的图像分类任务，并横向对比经典卷积神经网络（CNN）与视觉变换器（Vision Transformer, ViT）在不同数据量级下的性能表现。为了全面评估模型的表现，我们选定了以下三个核心评价指标：

- **分类准确率 (Accuracy)**：衡量模型在测试集上预测正确的样本比例。
- **宏观 F1 分数 (Macro-F1)**：综合考量模型在 10 个类别上的精确率与召回率表现，避免类别不平衡带来的评估偏差。
- **训练效率 (Time/epoch)**：记录单轮训练（Epoch）所需的时间，用于评估模型的计算开销与工程可行性。

1.2 数据说明与预处理

实验采用 64×64 高分辨率版本的 CIFAR-10 数据集，包含 10 个常见物体类别，总计划分为 50,000 张训练集与 10,000 张测试集。在数据预处理与增强阶段，我们执行了以下操作：

1. **归一化 (Normalization)**：使用标准均值 `[0.4914, 0.4822, 0.4465]` 和标准差 `[0.2023, 0.1994, 0.2010]` 进行数据标准化。
2. **数据增强 (Data Augmentation)**：仅针对训练集启用了随机水平翻转（RandomHorizontalFlip），以提升模型的泛化能力。
3. **子集采样 (Subset Sampling)**：针对后续的小数据退化实验，我们在 `dataset.py` 中构建了基于 `numpy.random.permutation` 的随机子集采样逻辑，可支持按照 20% 与 10% 比例进行无偏采样。

2 模型设计

为了实现公平的横向对比，我们在设计上严格控制了 CNN 与 ViT 的参数量级（Params），确保两者在相近的模型容量下进行学习能力的对标。

2.1 卷积神经网络 (CNN) 架构

基线 CNN 模型自行设计了一个 4 层深度的卷积神经网络架构，其结构特点如下：

- **通道递增**：网络特征通道数随深度依次翻倍（64 → 128 → 256 → 512）。
- **卷积块设计**:每个特征提取阶段采用标准的 `Conv-BN-ReLU-Conv-BN-ReLU-MaxPool-Dropout` 现代特征提取范式。
- **分类头**：摒弃了传统大参数量的全连接层展平方式，采用全局平均池化层（Global Average Pooling, GAP），将特征图直接压缩至一维向量，随后送入带有 Dropout 的两层 MLP 分类器，有效减少了参数冗余。

2.2 视觉变换器 (ViT) 架构

ViT 模型完全基于 PyTorch 从零手写搭建，其核心超参数配置如表 1 所示。模型采用了 Pre-LN 结构的 Transformer Block，引入了随机深度（Stochastic Depth / DropPath）以增强网络深层训练的稳定性，并通过可学习的 CLS Token 汇总全局特征进行分类。

表 1: Vision Transformer (ViT) 核心超参数配置表

超参数名称	设定值	超参数名称	设定值
输入图像尺寸 (img_size)	64×64	注意力头数 (num_heads)	8
分块尺寸 (patch_size)	8×8	MLP 扩展比例 (mlp_ratio)	4.0
Patch 数量 (num_patches)	64	Dropout 概率 (dropout)	0.1
嵌入维度 (embed_dim)	256	DropPath 概率	0.1
Transformer 层数 (depth)	6	分类类别数 (num_classes)	10

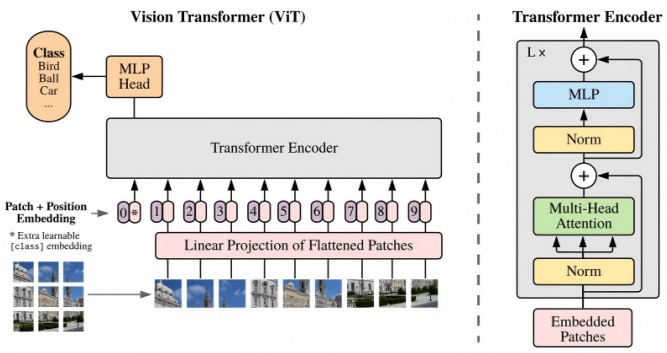

图 1: Vision Transformer (ViT) 整体架构示意图

3 实验结果与分析

3.1 多维度性能指标汇总

我们完成了针对 CNN 与 ViT 在三档数据比例（10%、20%、100%）下的完整训练测试。实验结果如表 2 所示，展现了两种架构在不同数据规模下的适应性差异。

表 2: CNN 与 ViT 在不同数据比例下的性能对比汇总

模型架构	数据比例	测试准确率 (Acc)	Macro-F1	单轮耗时 (s)
CNN	10%	70.95%	0.7090	18.42
(ResNet-18)	20%	79.89%	0.7986	24.53
	100%	90.41%	0.9038	61.30
ViT	10%	56.37%	0.5600	28.05
	20%	63.31%	0.6300	41.95
	100%	80.71%	0.8059	152.82

从表 2 的结果可以清晰看出两种架构的特性差异。在 10% 与 20% 的极小数据场景下，CNN 凭借卷积核天然的平移不变性与局部感知（归纳偏置），达到了 70.95% 与 79.89% 的准确率；而同等数据量下，ViT 仅有 56.37% 与 63.31%，出现了严重的性能退化。这有力地证明了全局自注意力机制在缺乏足够数据驱动时，极难自发学到有用的图像空间结构。但随着数据量扩展至 100%，ViT 的准确率获得了近 24 个百分点的巨大增幅（跃升至 80.71%），展现出惊人的数据扩展潜力。

3.2 训练曲线对比分析（数据效率实验）

为深入探究 CNN 与 ViT 在不同数据规模下的收敛特性，我们生成了包含左右双子图的综合对比图（见图 2）。左侧子图展示了训练损失（Loss）的下降趋势，右侧子图展示了测试准确率（Accuracy）的演进过程。

通过颜色与线型的多维编码，图表清晰地呈现了实验规律：

- **颜色编码（架构区别）**：蓝色系代表 CNN（ResNet-18），橙/红色系代表 ViT。
- **线型编码（数据比例）**：
  - **10% 数据量**：浅色 + 点虚线 (:)

– **20% 数据量：**中等色 + 虚线 (--)

– **100% 数据量：**深色加粗 + 实线 (-)

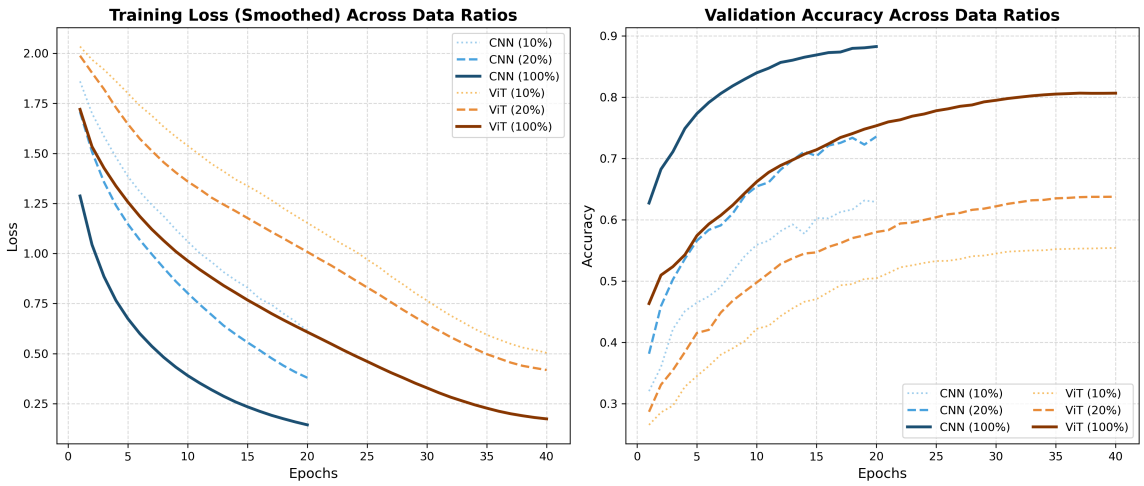

图 2: CNN 与 ViT 在 0.1、0.2、1.0 三档数据比例下的训练曲线对比。左图：训练损失曲线；右图：测试集准确率曲线。

3.2.1 结果分析：归纳偏置与 Scaling Law 的博弈

根据表 2 的量化指标与图 2 的可视化曲线，我们可以得出以下关键发现：

1. **CNN 的小数据稳健性：**在 10% 数据（5,000 张）极度匮乏的情况下，CNN 依然能达到 70.95% 的准确率，而 ViT 仅为 56.37% 。这证明了卷积操作自带的局部平移不变性等“归纳偏置”在小样本学习中起到了决定性的先验辅助作用。

2. **ViT 的高上限与 Scaling 性质：**随着数据比例从 0.1 提升至 1.0，ViT 的准确率从 56.37% 巨幅提升至 80.71% ，表现出比 CNN 更强的“数据饥渴”特性和性能天花板。

3. **收敛稳定性：**观察 Loss 曲线可以发现，ViT 的曲线在训练初期波动较大，且对学习率（LR）及 Warmup 策略更为敏感；相比之下，CNN 的蓝色实线在各数据量下均表现出更快的下降速度与更平滑的收敛路径。

3.3 ViT 模型 Attention Map 可视化

为探究 ViT 模型对图像特征的全局感知与捕捉能力，我们提取了 Transformer 编码器层的 Attention Weights，并针对验证集样本生成了热力图。图 3 展示了 6 个不同样本的注意力集中区域，红色/高亮部分代表分类 Token（CLS）在决策时主要聚焦的输入 Patch。

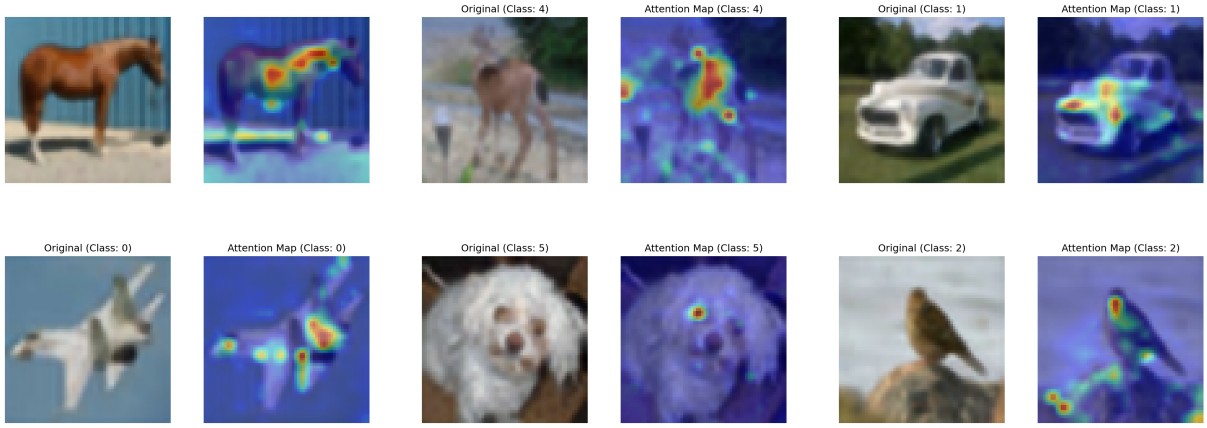

图 3: ViT 模型的 Attention Map 热力图可视化。高亮区域代表模型主要关注的特征区域。

3.4 速度与精度权衡 (Speed-Accuracy Trade-off)

为直观展示不同模型架构及数据规模下的计算开销与分类性能关系，我们绘制了速度-精度权衡散点图。不同实验设置（模型架构 × 数据量比例）将作为独立的数据点呈现，从而综合评估 ViT 与 CNN 的工程部署性价比。

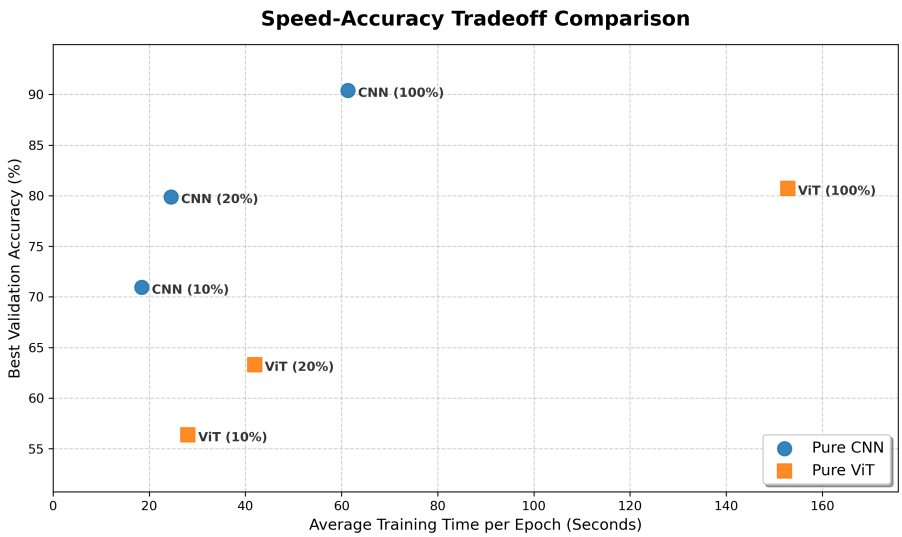

图 4: 模型速度与精度权衡图 (Speed-Accuracy Trade-off)。横轴为平均单轮训练时间（秒/Epoch），纵轴为测试集最佳准确率（Accuracy）。不同颜色的点代表不同的模型架构与数据量组合。

4 后续计划与进度安排

得益于项目前期的顺利推进，我们在第 5-6 周已超额完成了核心架构的搭建与全部三档数据比例（10%、20%、100%）的模型训练。因此，期末最终报告前的重点将从“跑通基线”全面转向“深度特性分析”与“工程化收尾”。具体进度安排调整如下：

时间节点	核心任务安排
第 7 - 8 周	**数据规模趋势图出图：**提取各比例下的最佳评测指标，绘制 Data Scale Trend 曲线，直观展现 ViT 与 CNN 随数据扩充的收益斜率博弈（Scaling Law）。 **深度错误分析 (Error Analysis)：**绘制测试集混淆矩阵（Confusion Matrix），探讨两种模型在相似类别（如猫与狗、卡车与汽车）上的错分偏好，并挑选典型案例进行针对性分析。
第 9 - 10 周	**工程化收尾与仓库打磨：**进行全局 Code Review，清理冗余输出；完善 `main.py` 参数解析，编写详尽的 `README.md` 与自动化复现脚本 `run.sh`，提升代码库的开源交付质量。 **期末报告撰写：**整合所有图表与讨论结论，完成最终结课报告排版，签署诚信声明，并完成 Git 代码仓库的封板提交。